

Emotion Detection & Learning Support Engine

AI Learning Assistant — Team SWTID-2026-7049

Project Description

The AI Learning Assistant reads a student's own written description of a study problem, detects the emotion behind it across five classes — Bored, Confident, Confused, Curious, and Frustrated — and responds with guidance adapted to both that emotion and the student's field of study, instead of the same flat response regardless of how the student is actually feeling.

Detection runs two independently-trained models in parallel: a lightweight BiLSTM and a fine-tuned BERT classifier. Rather than picking a single "winner" model by assumption, a rule-based decision engine — calibrated against a 770-row validation analysis, not intuition — decides which prediction to trust and how much, returning a final emotion, a HIGH/MEDIUM/LOW trust level, and a machine-readable reason code. That decision, together with the student's field of study, conditions a prompt sent to an LLM (Gemini, with OpenRouter as an alternate provider) for the final supportive response, with a deterministic template fallback whenever AI generation is turned off or unavailable.

Every interaction can optionally be logged to CSV and reviewed later through a Plotly-based analytics dashboard covering emotion distribution, an emotional-journey timeline, and a per-field breakdown.

Note: this project does not use a database — interaction history is CSV-based only, so no ER diagram is included.

Scenarios

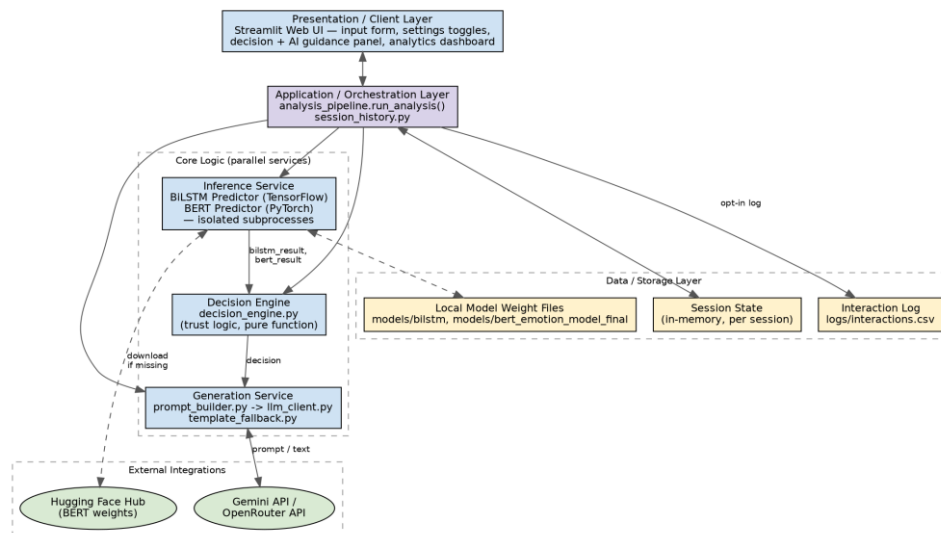
Scenario 1 — High-trust agreement: A student studying Physics writes "What is inertial momentum? Is it the same as normal momentum? I am very confused." BiLSTM's top pick is a close Confused/Curious mix (47.2%/46.3%), but BERT is highly confident in Confused (98.5%). Because BERT's confidence clears the trust threshold and both models agree on the same label, the decision engine returns Confused at HIGH trust (reason code `agreement_high_confidence`), and the AI response walks through the specific confusion between inertia and momentum.

Scenario 2 — Medium-trust disagreement: A student studying Computer Science writes "I don't know how recursion works, but I am curious!" Here BiLSTM's top pick is Confused (69.1%) while BERT confidently says Curious (98.4%) — the two models disagree, but BERT's confidence is still high. The decision engine follows BERT (which is right far more often than BiLSTM on disagreements, per the validation analysis) but flags the result as MEDIUM trust (reason code `disagreement_high_confidence`) rather than presenting it with unearned certainty. The response leans into the curiosity, offering a next step rather than just reassurance.

Scenario 3 — Low-trust fallback: When BERT is unavailable (e.g. on the degraded public deployment) or its confidence falls below the 0.8 calibration threshold, the decision engine reports LOW trust regardless of what BiLSTM says, since BiLSTM alone is only ~53% accurate and BERT's own confidence is close to a coin-flip below that threshold. This branch is exercised automatically any time BERT can't be loaded — the app degrades gracefully instead of failing.

Technical Architecture

The application is a layered Streamlit app: a presentation layer for input/settings/results/analytics, an orchestration layer that sequences one Analyze click end-to-end, two parallel core services (inference and, downstream, decision + generation), external integrations (the LLM provider and, when needed, the Hugging Face Hub for BERT weights), and a data/storage layer (session state, local model files, an opt-in CSV log).



Layered solution architecture: Presentation → Orchestration → Inference/Decision/Generation → External Integrations & Data

TensorFlow and PyTorch/Transformers cannot coexist in the same process — confirmed to cause a hard segfault during development — so each model loads in its own isolated subprocess, verified through dedicated isolation tests (envtesting/test_isolation_minimal.py, test_isolation_realistic.py, 3/3 passing locally).

Prerequisites

- Python 3.12 (Python 3.14 was found incompatible with TensorFlow 2.16.1)
- Familiarity with Streamlit for the UI layer
- Familiarity with TensorFlow (BiLSTM) and PyTorch/Transformers (BERT)
- A Gemini API key (<https://aistudio.google.com/>) and/or an OpenRouter API key
- Git, for version control

Project Workflow

The build proceeded epic-by-epic over 46 commits across 5–14 July 2026 (reconstructed from the project's real git history — see the Project Planning document for the full backlog). Six milestones, each corresponding to one completed epic:

Milestone 1: Development Environment

- Set up a dedicated Python 3.12 virtual environment, downgrading from Python 3.14 after it broke TensorFlow compatibility.
- Installed CUDA-enabled PyTorch (cu128) for BERT fine-tuning and CPU-only TensorFlow for BiLSTM.
- Verified GPU acceleration (RTX 5070 Ti) and confirmed the two frameworks could coexist safely — later hardened into full subprocess isolation once same-process loading was confirmed to segfault.

Milestone 2: Dataset Preparation & Model Training

- Selected and merged GoEmotions, EmpatheticDialogues, and ISEAR into a unified 5-class label set.
- Found the Bored class significantly underrepresented; chose class weighting over oversampling or synthetic generation, to preserve all classes without inflating overfitting risk.
- Trained a BiLSTM classifier with class weighting, intentionally on CPU (a Ryzen 9 9900X outpaced waiting on Kaggle, and TensorFlow dropped native Windows GPU support after v2.10).

- Applied domain-adaptive fine-tuning ahead of the main BERT pass, then fine-tuned a BERT classifier (GPU-accelerated) with class weighting and keyword adjustments.
- Verified both trained models load correctly and agree on label ordering before integration (models/integration_readiness_report.txt — verdict: READY).

Milestone 3: Inference & Persistence Pipeline

- Built shared text preprocessing with keyword enhancement for both models.
- Wrapped BiLSTM and BERT behind a common 5-class prediction interface.
- Added mixed-emotion detection, so a single dominant label isn't forced when two emotions are close.
- Standardized both models' outputs into one unified prediction schema.
- Added cached model loading (once per session) and CSV-based interaction persistence.

Milestone 4: AI Response Generation & Decision Engine

- Captured field + problem input and built a Gemini prompt incorporating the detected emotion and confidence.
- Generated empathetic, field-aware responses, with a template-based fallback when AI is disabled or fails.
- Made responses regenerate correctly when the AI toggle changes mid-session, keeping model scores in sync.
- Added session history for tracking interactions across a session.
- Self-directed, outside the formal epic plan: built the data-driven decision engine described above, after analyzing the 770-row validation set to find BERT alone reaches 83% accuracy vs. BiLSTM's 53%, and that BiLSTM works better as a secondary confidence signal than an equal vote.

Milestone 5: Streamlit UI

- Implemented the base layout and session state.
- Built sections for field selection, problem input, model comparison, mixed-emotion display, confidence bars, and analytics tabs.
- Added form controls, a working clear/reset action, spinners, and graceful error handling.
- Visualized scores with cached, Plotly-based charts — caching only the data transforms, not the Figure objects, after a rerun-triggered white-screen crash was traced to stale cached Figures being reused.

Milestone 6: Validation & Deployment Readiness

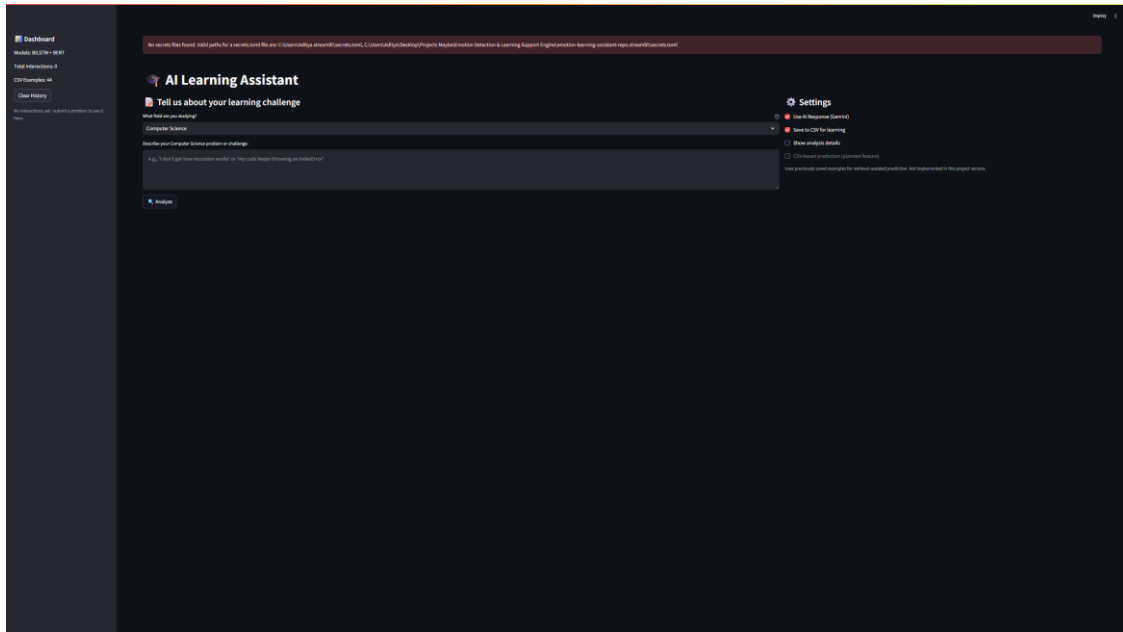
- Validated the full UI flow end-to-end: a 10-test automated Streamlit AppTest suite (real weights, real LLM calls) plus a manual click-through pass — 10/10 passing.
- Chose Streamlit Community Cloud for deployment; added DISABLE_BERT / DISABLE_AI_RESPONSE escape hatches after finding an unresolved OpenRouter-call hang specific to that environment, and shipped the public app in a deliberately reduced (BiLSTM-only, template-response) but fully stable mode.

Exploring the Website's Web Pages

All screenshots below were taken running the app locally in full mode (both models + real AI). The red banner visible in each ("No secrets files found...") is an expected, harmless warning — local development uses a .env file rather than Streamlit's secrets.toml, and app.py's own secrets-loading code already wraps this in a try/except for exactly this case.

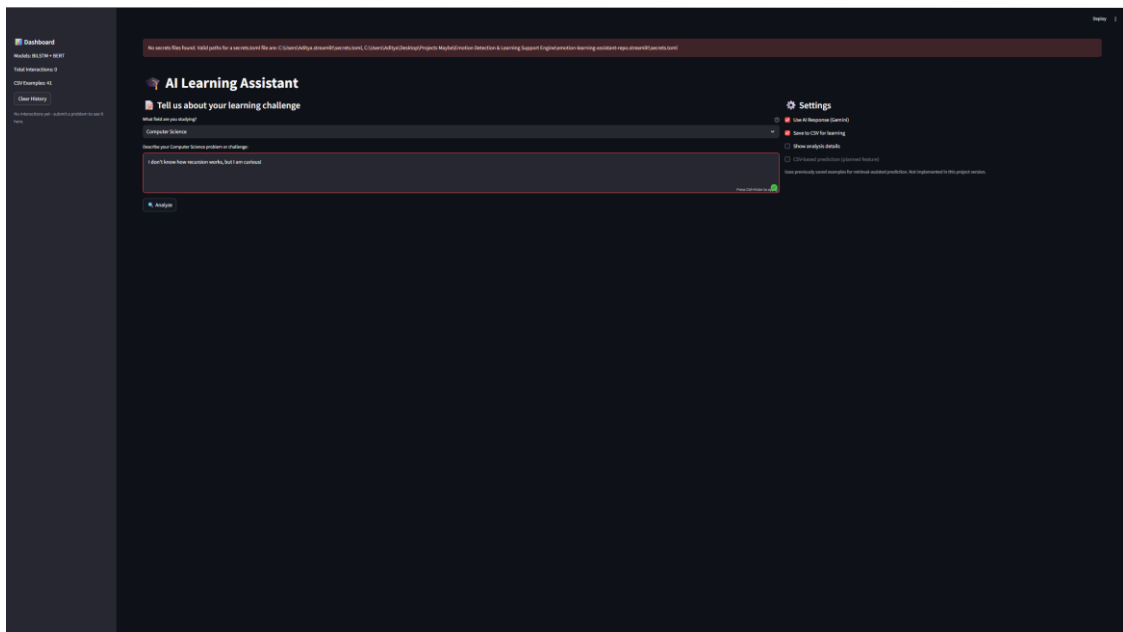
Landing Page

The sidebar shows live model status (BiLSTM + BERT ready) and session stats. The main panel holds the field selector, problem description box, and the Settings panel with independent toggles for AI response generation, CSV saving, detail visibility, and the clearly-labeled, disabled "CSV-based prediction (planned feature)" option.



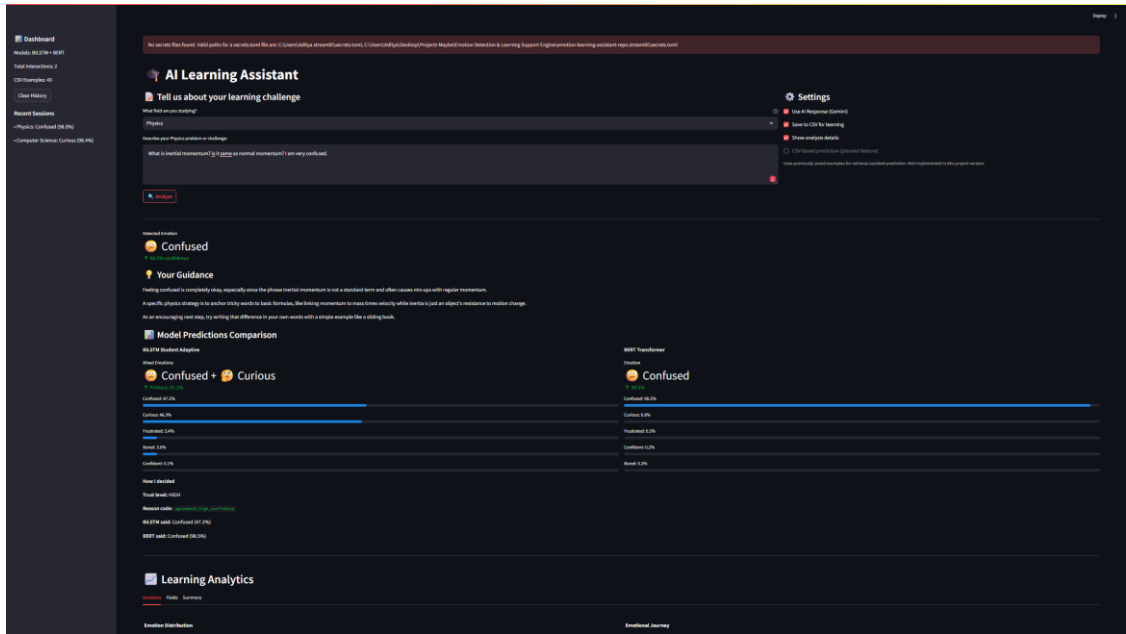
Input Ready to Submit

A field is selected and a problem described; the textarea is focused and ready for Ctrl+Enter or a click on Analyze.



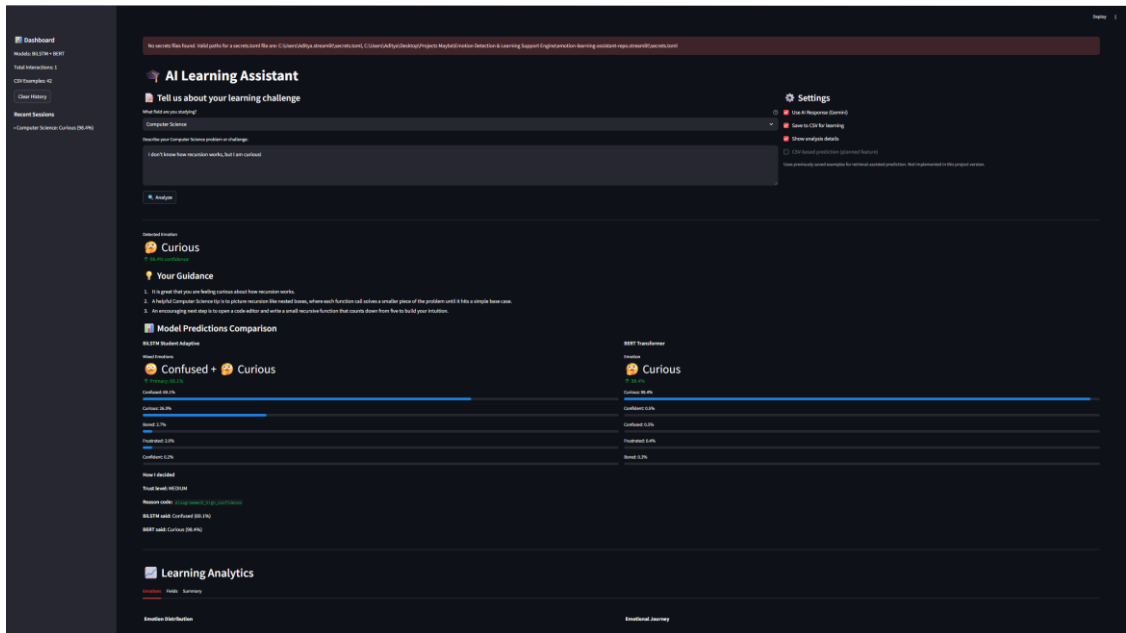
Results — High-Trust Agreement

Corresponds to Scenario 1 above. Decision Summary, AI Guidance, the side-by-side Model Predictions Comparison, and the "How I decided" explainability panel (Trust level: HIGH, reason: agreement_high_confidence) are all visible with Show analysis details switched on.



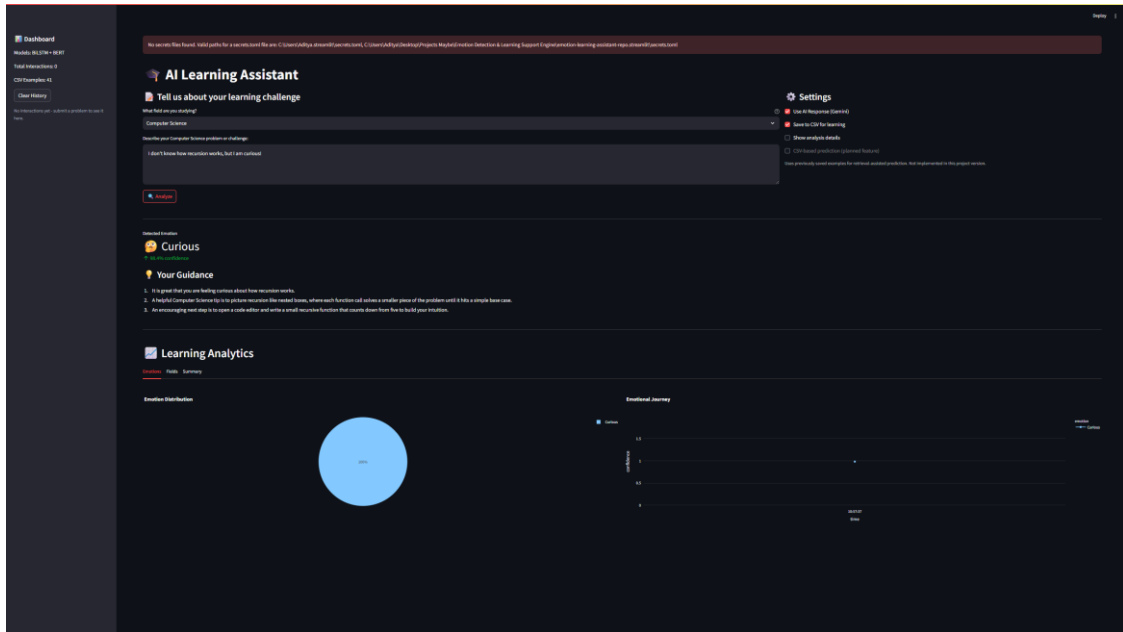
Results — Medium-Trust Disagreement

Corresponds to Scenario 2 above. The two models disagree (BiLSTM: Confused 69.1%, BERT: Curious 98.4%); the decision engine follows BERT but honestly reports MEDIUM trust (reason: disagreement_high_confidence) instead of overstating certainty.



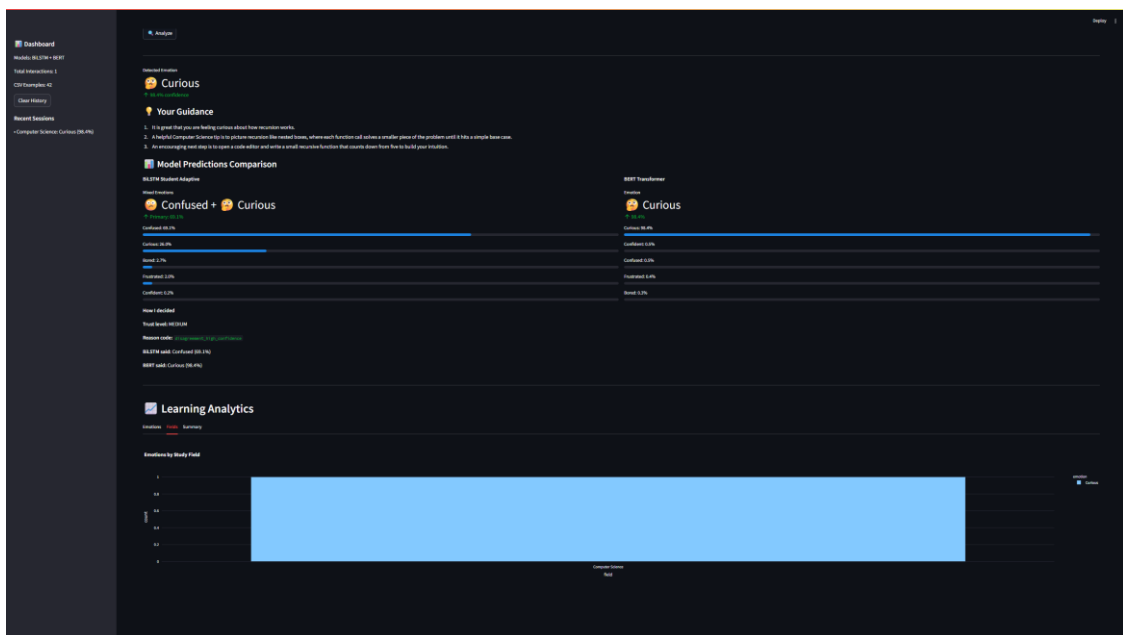
Analytics Dashboard — Emotions Tab

The Emotions tab's pie chart and emotional-journey timeline, populated from session history.



Analytics Dashboard — Fields Tab

The Fields tab's bar chart, breaking down detected emotions by study field.



Known Issues / Limitations

- The public deployment (<https://emotionalgemini.streamlit.app/>) runs BiLSTM-only with template responses, not the full pipeline shown above — a known, disclosed tradeoff, not a defect. Full behavior is verified locally (see `VALIDATION_REPORT.md`, `PERFORMANCE_TESTING.md`).
- CSV-based retrieval prediction and a bring-your-own API key were both scoped out for this submission — both are visible in the UI or backlog as clearly labeled, deferred items rather than silently dropped.
- The Analytics "Summary" tab is intentionally empty; no scope has been defined for it yet.

Conclusion

The AI Learning Assistant demonstrates that emotion-aware tutoring support can be built on genuinely data-driven foundations rather than assumption: the dual-model design, the trust/decision logic, and even the public deployment's reduced-mode configuration are all backed by real validation numbers (BERT 83% / BiLSTM 53% accuracy on a 770-row set, 0% failure rate across 367 load-test requests) rather than guesses. The project also demonstrates disciplined engineering judgement in what wasn't built: CSV-based retrieval prediction and a bring-your-own API key were both evaluated and consciously deferred once their real costs became clear, and both remain honestly labeled in the running app rather than hidden. Looking ahead, natural next steps include resolving the OpenRouter hang blocking full-mode public deployment, defining scope for the Analytics Summary tab, and revisiting the deferred features once their groundwork (a retrieval pipeline; refactored, non-singleton LLM clients) is in place.